
The iOS build — what it needs to work exactly like Android

What the iPhone build needs to work exactly like Android — where it stood, what is done in the repository and proven on GitHub's Mac, the eight things only the client's Apple and Google accounts can supply, and what remains before the App Store.

Aajoo Homes — the iOS build

Prepared 15 September 2026 by the ZypheX Tech development team

Compiled unsigned on GitHub's macOS runner, 15 September 2026 (run 34996287126)

15 September 2026. The client asked for the app on iPhone, working exactly as it does on Android. This is the whole picture: where the iOS half stood this morning, what has been done in the repository today, what the build machinery now does, what only the client's accounts can supply, and what remains before an App Store submission.

The one hard fact first: **an iOS app can only be compiled on a Mac.** There is none on the team's desks, and Windows cannot run Xcode or the iOS Simulator. So the compile runs on GitHub's macOS runners (free, because this repository is public), every time the app changes — and the first build on a real iPhone goes through Apple's TestFlight, which needs the client's Apple Developer account. Nothing in this document is blocked on buying a Mac.

1. Where iOS stood this morning

The `ios/` folder was the untouched Flutter template from the day the project was created:

	Android (build 94)	iOS (this morning)
App name on the phone	Aajoo Homes	"Rent Home"
Icon	Aajoo logo	Flutter's blue logo
Launch screen	Logo on white	Blank
Minimum OS	Android 5+ (API 21)	iOS 12.0 — below what four of the plugins need
Location permission text	(Android needs none)	Missing — iOS refuses to prompt without it
Photo-library permission text	(none)	Missing
Google Maps key	In the manifest	None — every map tile blank
Google Sign-In callback	Configured	None — the Google sheet would never return to the app
Push (Firebase)	Works	No APNs entitlement, no background mode
Tap-to-dial / WhatsApp / directions	Declared in <code><queries></code>	Not declared — <code>canOpenURL</code> answers false, taps do nothing
Razorpay UPI intent	Works	UPI app schemes not declared — the UPI-app list would be empty
permission_handler	Works	Every permission compiled out — requests answer "denied" without asking
Privacy manifest	n/a	None (App Store requires one since 2024)
Build script / verifier / CI	<code>build_release.ps1</code> , <code>verify_release_apk.py</code>	None

None of the Dart code needed changing for iOS except one place (push, §2.4). The gaps were all in the iOS project itself.

2. What is done now, in the repository

2.1 The iOS project (`aajoo_app_2026/ios/`)

- `Runner/Info.plist` — display name **Aajoo Homes**; usage strings for location (while in use), photo library, camera and microphone; `UIBackgroundModes: remote-notification` for push; `LSApplicationQueriesSchemes` with exactly the schemes the Dart code launches (`tel`, `mailto`, `whatsapp`, `comgooglemaps`, `maps`) plus the UPI apps Razorpay lists (`phonepe`, `paytmmp`, `tez`, `gpay`, `bhim`, `upi`, `credpay`); `ITSAppUsesNonExemptEncryption = false` so TestFlight uploads do not stop to ask; and two **placeholders** that only the client's accounts can fill — `GMSApiKey` (the iOS Maps key) and `GIDClientID` + its reversed form as the URL scheme (Google Sign-In). The release verifier refuses a build that still carries a placeholder.
- `Runner/AppDelegate.swift` — hands the Maps key from Info.plist to `GMSServices` before any map is built (logs loudly if it is the placeholder), registers the plugins, and sets the notification-centre delegate so a tapped notification reaches Firebase.
- `Podfile` — `platform :ios, '14.0'` (the floor `google_maps_flutter_ios` 2.13 and the Firebase iOS SDK need; every pod is lifted to it), and the `permission_handler` compile flags for the permissions the app actually asks for: notifications, location (while in use), photos, camera.
- `Runner.xcodeproj` — deployment target 12.0 → **14.0** in all three configurations; `Runner.entitlements` (`aps-environment` for push) wired in as `CODE_SIGN_ENTITLEMENTS`; `PrivacyInfo.xcprivacy` added to the Runner group and the Resources phase. `Flutter/AppFrameworkInfo.plist` minimum OS 14.0.
- `Runner/PrivacyInfo.xcprivacy` — the App Store privacy manifest: no tracking, no tracking domains, the one "accessed API" category Flutter's own plumbing uses (UserDefaults, reason CA92.1). The plugins ship their own manifests.
- **App icon** — the full iPhone/iPad/marketing set (19 files) generated from the same `ic_launcher-web.png` the Android launcher uses, flattened on the same `#FAF8F4` (iOS icons must be opaque). *The 1024-px marketing icon is upscaled from the 512-px master; a 1024-px original from the designer would be sharper — ask.*
- **Launch screen** — the Android `launch_image.png` at 1×/2×/3× in `LaunchImage.imageset`, centred on white, exactly as `launch_background.xml` does.

2.2 Build, verify, CI

- `tool/build_ios.sh` — the macOS twin of `build_release.ps1`: the same `--dart-define` set (`API_BASE_URL`, `RAZORPAY_KEY`, `APP_VERSION`, `ALLOW_TEST_PAYMENTS`), the same refusals (no http endpoint, no dev endpoint or sandbox key without the flag), `--no-codesign` for a compile check or `--export <ExportOptions.plist>` for a signed `.ipa`.
- `tool/verify_release_ipa.py` — the twin of `verify_release_apk.py`: opens an `.ipa`, a zipped `Runner.app` or the `Runner.app` folder; checks the bundle id is `com.aajoo.aajoohomes`, the display name, that the permission strings are present, that no placeholder is left in Info.plist, that the version in Info.plist and in the Dart code is the one the build claims, and the same forbidden-string, plain-http and endpoint checks as the APK verifier. Exercised against a synthetic `Runner.app` on Windows (fails with the placeholders, passes with `--allow-placeholders`).

- `.github/workflows/ios-build.yml` — two jobs on `macos-15`:
 1. **compile** — on every push that touches the app: Flutter 3.44.0, `flutter test`, `tool/build_ios.sh ... --no-codesign`, the verifier with `--allow-placeholders`, and the unsigned `Runner.app` uploaded as an artifact. **Needs no Apple account.** This is what proves the iOS half builds.
 2. **testflight** — run by hand (`workflow_dispatch` → "testflight"): imports the client's signing certificate and provisioning profile from repository secrets, fills the two placeholders from secrets, builds a signed `.ipa`, verifies it (no placeholders allowed), and uploads it to TestFlight. It stops with a named list of any secret that is missing.

2.3 Dart

- `NotificationService` — on iOS, asks Firebase to register with APNs (`requestPermission`) and waits for the APNs token before asking for the FCM token, because `getToken()` on iOS throws "apns-token-not-set" if called first — and on the Simulator, which has no APNs, it would throw forever; push is simply off there. The `token!` that followed used to throw inside the permission flow and take sign-in down with it; a missing token is now a logged "no push this run".
- Foreground pushes — iOS shows nothing for a push that arrives while the app is open unless told to present it; the local notification now carries `DarwinNotificationDetails(presentAlert, presentBadge, presentSound)`, so the banner Android shows, iOS shows too.
- Everything else was already platform-aware: `stay_map` offers Apple Maps beside Google Maps on iOS; `firebase_options.dart` already carried the iOS app's Firebase values (`1:1006999733744:ios:...`, bundle `com.aajoo.aajoohomes`), so the iOS app is registered in the Firebase project.

2.4 Backend

- `utils/methods.js` `sendPushNotification` — every push now carries an `apns` payload (`sound: default`, `content-available: 1`, priority 10). Without it the same message that banners on Android arrives on an iPhone silent. Pinned by `tests/aPushReachesAnIphone.test.js`.

3. What only the client can do

Everything below is an account the client owns. None of it is code, and the CI is written to take each value as a repository secret so nothing sensitive is ever committed.

#	What	Where	Why
1	Enrol in the Apple Developer Program (₹9,000-odd a year) as the company	<code>developer.apple.com</code>	Nothing runs on a real iPhone, and nothing reaches TestFlight or the App Store, without it. If Aajoo is a registered company, enrol as an <i>Organisation</i> (needs a D-U-N-S number) so the App Store listing carries the company name, not a person's

#	What	Where	Why
2	Create the App ID <code>com.aajoo.aajoohomes</code> with Push Notifications on	Certificates, Identifiers & Profiles	The <code>aps-environment</code> entitlement fails to sign without it
3	Create an APNs authentication key (.p8) and upload it to Firebase → Project settings → Cloud Messaging → iOS app	developer.apple.com → Keys; console.firebase.google.com	Firebase cannot deliver a single push to an iPhone without it
4	Download <code>GoogleService-Info.plist</code> for the iOS app from the Firebase console	Firebase → Project settings → Your apps → iOS	It carries the iOS OAuth client (<code>CLIENT_ID</code> , <code>REVERSED_CLIENT_ID</code>) Google Sign-In needs. If the iOS app has no OAuth client yet, enabling Google sign-in for it in Firebase Authentication creates one
5	Create an iOS-restricted Google Maps key in the same Google Cloud project, restricted to bundle id <code>com.aajoo.aajoohomes</code> , with <i>Maps SDK for iOS</i> enabled	console.cloud.google.com → APIs & Services → Credentials	The Android key is restricted to the Android app and its signing certificate; iOS will be refused with it. Add it to the same budget alert as the others (\$2.6 of the master list)
6	Create the app record in App Store Connect (name Aajoo Homes, bundle id above, primary language English (India)) and an internal TestFlight group with the tester's Apple ID	appstoreconnect.apple.com	TestFlight is the only way to put a build on a tester's iPhone
7	Create an App Store Connect API key (role App Manager) — note the Key ID and Issuer ID, download the <code>.p8</code> once	App Store Connect → Users and Access → Integrations	The CI uploads to TestFlight with it, without anyone's Apple password
8	Issue a distribution certificate and an App Store provisioning profile for the App ID	Certificates, Identifiers & Profiles	The CI signs with them. Export the certificate with its private key as a <code>.p12</code>

The repository secrets the TestFlight job reads

Secret	Value
<code>APPLE_TEAM_ID</code>	The 10-character Team ID from the developer account
<code>IOS_CERTIFICATE_P12_BASE64</code>	The distribution certificate + key, base64 of the <code>.p12</code>
<code>IOS_CERTIFICATE_PASSWORD</code>	The <code>.p12</code> password
<code>IOS_PROVISIONING_PROFILE_BASE64</code>	base64 of the <code>.mobileprovision</code>
<code>IOS_PROVISIONING_PROFILE_NAME</code>	The profile's name as shown in the portal
<code>APP_STORE_CONNECT_ISSUER_ID</code> , <code>APP_STORE_CONNECT_API_KEY_ID</code> , <code>APP_STORE_CONNECT_API_KEY_P8</code>	The API key from step 7 (the <code>.p8</code> contents as-is)

Secret	Value
<code>IOS_MAPS_KEY</code>	Step 5
<code>IOS_GOOGLE_CLIENT_ID</code>	<code>CLIENT_ID</code> from step 4 (ends in <code>.apps.googleusercontent.com</code>)
<code>IOS_GOOGLE_SERVICE_INFO_PLIST_BASE64</code>	base64 of the file from step 4 (optional — the app already carries the Firebase values in <code>firebase_options.dart</code> ; the plist is belt and braces)

Base64 on a Mac: `base64 -i file | pbcopy`. On Windows:

`[Convert]::ToBase64String([IO.File]::ReadAllBytes("file")) | Set-Clipboard`.

4. What remains before the App Store

- 1. Sign in with Apple.** App Store Review Guideline 4.8: an app that offers a third-party login (Google) must also offer a login that limits data collection, lets the user hide their email, and does not track — Sign in with Apple is the one Apple accepts without argument. This is a real work item on both sides: the app (`sign_in_with_apple`, the button beside Google's, the Apple provider in Firebase Auth) and the backend (verify Apple's identity token, create/link the account the way the Google path does — see the Google-account logout note in the master list, the same care applies). Not needed for TestFlight; **needed for App Review**. Estimate: a day, plus the client enabling the capability on the App ID.
- 2. The on-device drive.** Every flow driven on the Android emulator this week has to be driven on an iPhone from TestFlight — the Android drives found five defects no test had, and iOS will have its own: sign-up (OTP, Google), the DOB picker, search and the map (Google tiles, the "near me" prompt), a listing's page and Things to know, the booking sheet (deposit and cleaning statements, the deal banner), Send an Offer for a night and for a week, the host side (accept / counter / decline, the popup), Razorpay checkout (card, and the UPI intent list), pay-at-property, My Bookings, notifications (banner in the foreground, tap from the background, cold start), the listing wizard (photo upload from the library, the document picker, the map pin), profile photo, share and print of an invoice, tap-to-dial and WhatsApp, sign-out and the keychain (see §5).
- 3. App Store Connect listing:** screenshots for 6.7" and 6.5" iPhones (and 12.9" iPad unless the app is marked iPhone-only — recommend iPhone-only for the first release), the privacy questionnaire (account, location while in use, photos the user uploads, payment handled by Razorpay), the support URL and privacy policy URL (both exist on the website), age rating, review notes with a test account and a test card.
- 4. A 1024-px icon from the designer** (the current one is an upscale of the 512-px Android master).

5. Differences a tester will see — by design, not defects

- Permissions are prompted by Apple, once.** Location and notifications each show Apple's sheet the first time; if refused, the app cannot ask again — it opens Settings (the app already handles `permanentlyDenied` that way).

- **No back button.** iOS navigates back by the edge swipe or the arrow at the top left; the Android system back is gone.
 - **Directions** offers *Open in Google Maps* and *Open in Apple Maps*; Android offers Google only.
 - **Razorpay's UPI list** shows the UPI apps installed on that iPhone (via the declared schemes); the card flow is identical.
 - **The keychain survives an uninstall.** `flutter_secure_storage` writes to the iOS keychain, which persists across delete-and-reinstall. A reinstalled app can therefore find a stale token from before. Sign-out clears it; a tester who deletes the app without signing out and sees themselves still signed in has met this, not a bug. (The same is true of the cached FCM token; the server prunes dead tokens.)
 - **Push is off on the Simulator** (no APNs). Only a real iPhone receives notifications.
 - **Version.** Settings shows `1.0.0`; the build number is internal on both platforms. Identify an iOS build by its TestFlight build number, which is the Android `versionCode` of the same commit.
-

6. The path to the first iPhone build

1. Push (done) → the **compile** job runs on GitHub and proves the iOS project builds. **Proven 15 September, run 34996287126:** 20 pods resolved, Xcode build in 240 s, the 472 tests green on the Mac, the verifier satisfied on Info.plist, the Runner binary and the Dart snapshot (bundle id, name, permission strings, the development endpoint and no other, version 1.0.0+94), and `Runner-1.0.0+94-unsigned.app.zip` (40.7 MB) kept as the run's artifact. (The first run, 34994667340, built the same app and then failed in the verifier itself, which had scanned forty SDK Info.plists and read three `http://` identifiers inside the Google, Firebase and Razorpay SDKs as endpoints — fixed the same hour.) Watch future runs at Actions → iOS build.
2. Client completes §3 (1–8) and adds the secrets.
3. Actions → iOS build → Run workflow → tick **testflight**. The job signs, verifies, uploads.
4. TestFlight processes the build (10–30 minutes); the tester installs the TestFlight app on their iPhone and accepts the invitation.
5. The drive in §4.2, with the same care as the Android drives, before anyone calls it a build.
6. Sign in with Apple (§4.1), then the App Store listing, then Review.

Two artifacts, one number: the iOS build of `1.0.0+N` is the Android build of `1.0.0+N`, from the same commit, with the same configuration.